

CHAPTER IV. MACHINE LEARNING & HYBRID MODELING – A WAY FORWARD

Classical macroscopic models, particularly those based on rational fraction representations, have been widely employed to describe microalgae growth and hybridoma cultures [215]. These models offer a structured approach that facilitates interpretability and theoretical consistency. They have been observed to perform adequately when addressing relatively simple representations; however, as the complexity of the system is augmented – particularly when multiple variables and inputs are involved – models become excessively stiff. Such stiffness has been found to hinder the capacity of these representations to capture the nuances of system dynamics with accuracy, leading to discrepancies between predicted and observed behaviors.

On the other hand, neural network models have been recognized for their flexibility in approximating complex and nonlinear relationships inherent in bioprocess systems. It has been noted, however, that this flexibility is accompanied by inherent challenges. In particular, the lack of embedded structural information may lead to the representation of noise as signal, overfitting to the training data, and limited generalization capabilities. These drawbacks become especially significant when the models are applied beyond the range of the operational conditions that have been previously explored.

To address these limitations, the integration of mechanistic structure and understanding with neural network flexibility represents a good trade-off solution for bioprocess modeling with hybrid modeling. This approach integrates fundamental mechanistic knowledge – especially concerning reaction kinetics – into the architecture of neural networks. By embedding the flexible framework of data-driven models within a basic and structured representation, hybrid methods aim to preserve the essential mechanistic insights while gaining from the adaptability of neural networks. Consequently, it is anticipated that such a model will improve performance in capturing complex system behaviors, while mitigating the risks of overfitting and ensuring a more reliable extrapolation beyond the immediate training domain.

1 BACKGROUND AND STATE OF THE ART

Hybrid modeling in bioprocess engineering originated as a means to overcome the limitations of both classical first-principles (mechanistic) models and purely data-driven approaches. Early seminal studies demonstrated that by incorporating a neural network element into the structure of the conventional mass-balance equations, one could capture unknown or highly nonlinear reaction kinetics while preserving physical interpretability [216], [217]. This “semiparametric” approach set the stage for later work that systematically combined a knowledge-based framework with machine learning tools [218].

The hybrid modeling paradigm has matured in two main directions. One line of research has focused on integrating feedforward neural networks – often with one hidden layer – with the first-principles description of bioreactors for applications such as state estimation, process monitoring, and control [219], [220]. Advances in optimization algorithms – such as the adaptive moment estimation (ADAM) method – have also enabled the transition toward deep hybrid models that use multilayer architectures [221].

A particular work with animal cells introduces and validates a hybrid modeling framework that combines the strengths of mechanistic (first-principles) models with the flexibility of neural networks [222]. In this approach, the inherently low quality and sparsity of measurement data – often encountered in biochemical applications – were addressed by embedding a neural network within the structure of a classical mass-balance model. Two alternative model structures were proposed: one in which the neural network approximated the reaction kinetics, and another in which it represented the complete reaction rate, including the pseudo-stoichiometric coefficients.

Microalgae cultivation is an attractive field for hybrid modeling. The complex interplay of light, temperature, and nutrient concentrations – among others – renders purely mechanistic models either too simplified or overly stiff. Recent reviews have highlighted machine learning approaches that complement traditional models by capturing nonlinear responses in microalgal growth [223].

Hybrid models in this area typically embed macroscopic mass balances – enforcing conservation laws – with neural networks trained on experimental data from photobioreactors. To initiate the exploration of hybrid modeling methods, the next section

presents applications to microalgae cultures: a Neural Network Model and an NN-based Hybrid Model.

2 SYNTHETIC DATA GENERATION AND MODEL TRAINING

At this stage, the objective is to evaluate the performance of these models. To achieve this, a set of synthetic data with realistic characteristics, which can be easily replicated under experimental conditions, is being utilized.

The training data is derived from the previously established, entirely physics-based microalgae culture model. The model is described by Equations 266-277, with the corresponding nomenclature and identified parameters provided in Table 2 and Table 10, respectively. Random white Gaussian noise is incorporated into the process and the measurements to simulate real-world variability:

$$\dot{x} = f(x, u) + v \quad 349$$

$$y = h(x, u) + w \quad 350$$

where v and w represent the process and measurement noise, respectively.

The process noise term can be expressed as $v = \sigma \eta$, where η is a standard normal variable $\eta \sim N(0,1)$, and σ is the standard deviation determining the noise level. The standard deviation is scaled based on the desired noise level relative to the typical magnitude of the function $f(x, u)$. An analogous formulation applies to the measurement noise term, although its magnitude is scaled with respect to the output.

In bioprocess modeling, typical process noise levels can be classified into low, medium, and high categories based on the magnitude of σ .

- For low process noise, the standard deviation is typically between 1-2% of $f(x, u)$, resulting in v values that, with 99.7% confidence, lie within ± 3 –6% of the nominal dynamics.
- For medium noise, the standard deviation is about 5-10% of $f(x, u)$, and the noise term v generally varies within ± 15 –30% of the nominal dynamics.

- High process noise, characterized by a standard deviation of approximately 20-30% of $f(x, u)$ – or higher, leads to v values typically falling within a range of ± 60 –90% of the nominal value.

Following these guidelines, for a typical magnitude of $f(x, u)$ equal to 1 unit, low noise levels would imply $\sigma \approx 0.01$ – 0.02 (with v between ± 0.03 and ± 0.06), medium noise levels would result in $\sigma \approx 0.05$ – 0.1 (with v between ± 0.15 and ± 0.3), and high noise levels would indicate $\sigma \approx 0.2$ – 0.3 (with v between ± 0.6 and ± 0.9). These values are empirical and should be adjusted to reflect the specific variability expected.

A medium level of process noise – i.e., $\sigma = 10\%$ of $f(x, u)$ – is used in the generation of synthetic data. This is intended to add to the process a higher degree of unpredictability, providing a tougher benchmark for the NN-based model to recognize and learn. Additionally, measurement noise is set to have $\sigma = 4\%$ of the output – i.e., $y(t)$.

The data is produced synthetically, incorporating randomly varying inputs that cover the full range of practical values. Initial conditions are also varied to span different operational areas and to exploit the system behavior, as detailed in Table 22. This approach ensures sufficient data is gathered to train the hybrid model while acknowledging that this type of system typically suffers from sparse measurements subject to noise.

A total of six batches are simulated; the first five are used for model training and internal validation via a k-fold procedure, while the final batch is reserved for cross-validation.

In all batches, the inflow substrate concentration is sustained at $25.0 \text{ g}_N/\text{L}$.

This entire study was conducted using *Python*, within the *Microsoft Visual Studio Code* environment. The main libraries utilized include *PyTorch*, a popular deep learning framework, for neural network implementation; and *SciPy*, for numerical computations and optimization tasks.

The frequency of measurement points is set at once per day. This is in line with some practical implications: biomass and substrate concentration readings can be collected online with plunging probes – as described in the previous sections; however, the substrate concentration probe is expensive to operate continuously. Nonetheless, the biomass concentration could be determined at a much higher frequency. Gaussian noise, with a

standard deviation equivalent to 4% of the mean value of the clean dataset, was added to the measurements. This method introduces proportional and realistic perturbations into the data.

Table 22. Initial Conditions, Input Variable Values, and Cultivation Time for Each Microalgae Continuous Culture Synthetic Data Runs.

Run	time [d]	Initial Conditions (x_0)			Input Variables (u_0)	
		X_0	Q_0	S_0	I_0	D
1	20	100	0.025	4.00	[200-800]	[0.100-0.350]
2	20	150	0.030	6.00	[200-800]	[0.100-0.350]
3	20	200	0.035	8.00	[200-800]	[0.100-0.350]
4	20	250	0.040	10.00	[200-800]	[0.100-0.350]
5	20	300	0.045	12.00	[200-800]	[0.100-0.350]
6	20	350	0.050	14.00	[200-800]	[0.100-0.350]

Note: Physical units are defined in Table 2.

Integration of the ordinary differential equations (ODEs), is performed utilizing the *solve_ivp* function from *SciPy*. By default, this algorithm employs the Runge-Kutta method – i.e., RK45. This method is an adaptive step size algorithm, which means it adjusts the step size during the integration to maintain accuracy.

However, there is an important consideration regarding the behavior of the solver and the repeated noise sampling. Since *solve_ivp* is primarily designed for deterministic ODEs, it evaluates the derivative function multiple times during each integration step. Each of these evaluations generates a new random noise sample. The adaptive stepping mechanism and internal interpolations of the solver may not propagate the noise in a consistent manner, potentially affecting the intended impact of the process noise. To address this issue, a fixed noise sample for each integration interval is imposed. Instead of allowing *solve_ivp* to resample the noise at every internal evaluation, a single noise vector should be generated for each integration step and maintained throughout the entire interval.

3 NEURAL NETWORK MODEL

The first and more intuitive approach is to replace the entire model with a neural network. Nonetheless, there is an important feature: the system is dynamic, while neural networks are typically static models. To bridge this gap, the role of the neural network is reframed, not to output the system states directly, but to produce their time derivatives instead. In this way, the model captures a series of discrete data that, later, using integration, allows for predicting the motion between them. By focusing on the time derivatives, the neural network becomes attuned to the system dynamics, effectively modeling how the states evolve rather than just where they are at a given moment.

$$\dot{\xi} = NN(\xi, u) \quad 351$$

In this model, the system states and inputs are provided as inputs to the neural network, NN . The network produces the temporal derivatives of the system states, $d\xi/dt$, as the output. This enables the equations to be integrated over time, resulting in the time evolution of the system states as the final output.

For the microalgae culture system, these equations can be described as follows:

$$\begin{bmatrix} \dot{X} \\ \dot{Q} \\ \dot{S} \end{bmatrix} = NN(X, Q, S, D, I_0) \quad 352$$

where $[X, Q, S, D, I_0]^T$ are the network inputs, and $[\dot{X}, \dot{Q}, \dot{S}]^T$ the network outputs.

Notice that the chlorophyll, Chl , is no longer present in this model. Its concentration is largely proportional to biomass levels, internal quota, and a term influenced by light irradiance – i.e., $Chl = \gamma(I^*)XQ$. Since its role in culture growth is inherently captured within the growth rate, explicit modeling of chlorophyll becomes unnecessary. In this framework, the specific growth rate neural network already incorporates X , Q , and I_0 as inputs, ensuring that the effects of light irradiance on growth rate are learned without direct dependence on chlorophyll concentration.

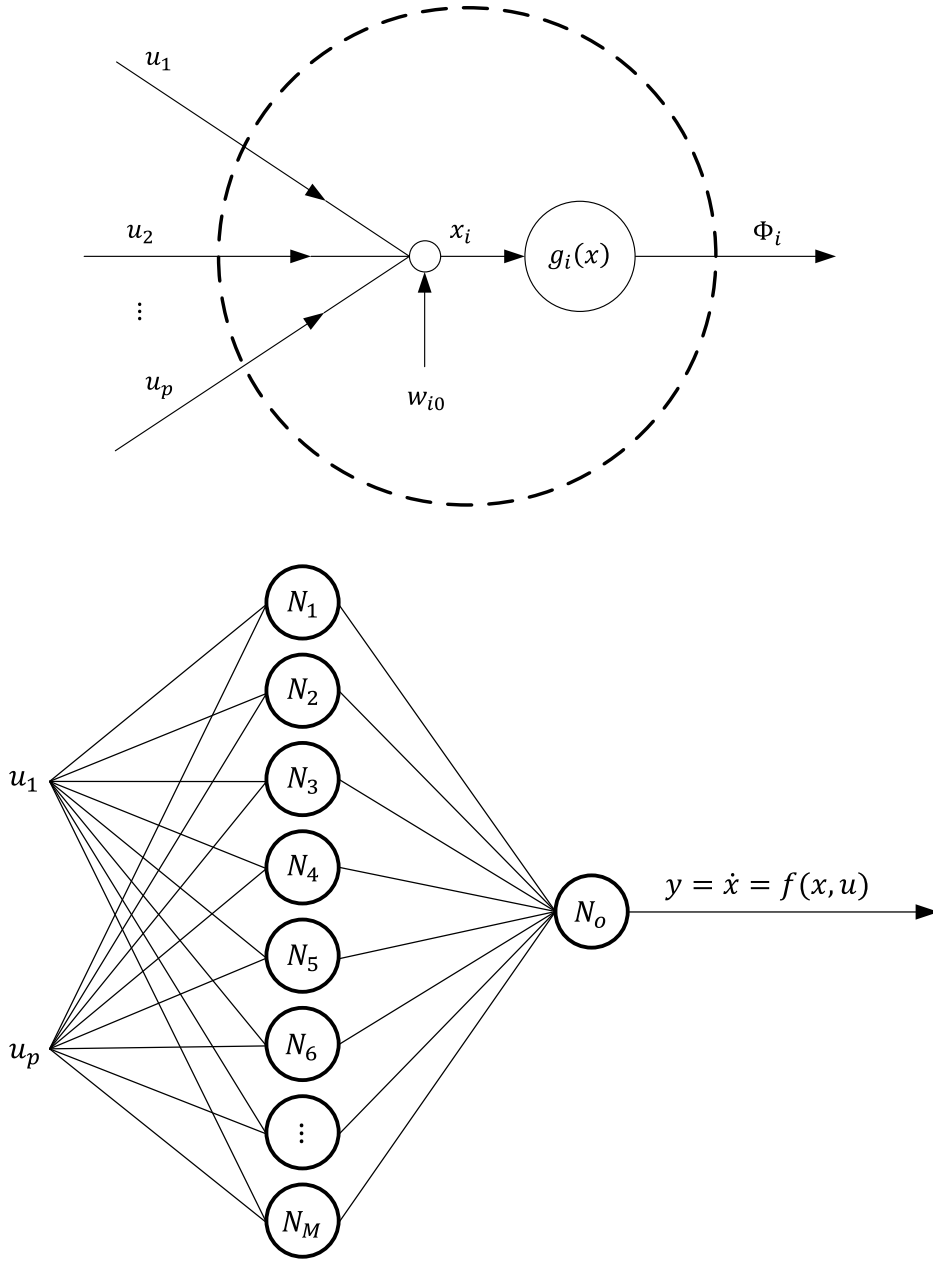


Figure 82. Neural Network Architecture and Neuron Internal Structure.

Composite view of a neural network. The network is depicted with a hidden layer containing M neurons, and a single output neuron, illustrating the overall connectivity and data flow within the system (below). The internal configuration of a representative neuron is also shown, detailing how its weighted inputs u_i , bias w_{i0} , and activation function $g_i(x)$ combine to produce an output Φ_i (above).

However, preliminary results show that this is not a robust and general solution. The model presented satisfactory behavior only as long as the light irradiance and dilution rate inputs were sustained at a constant value throughout each batch. It becomes apparent that data scarcity and measurement noise do not allow for proper model training when the system presents some complex and intricate relationships between variables.

To address these issues, the next section presents a hybrid model that integrates a general mass balance framework to provide structural coherence while exploiting the potential of neural networks to capture the complexities of kinetic variables.

4 NN-BASED HYBRID MODEL

In this new approach, a mechanistic model is introduced to provide the general framework of the model behavior, something that will grant some basic structure. In particular, this consists of the mass balance equations involving the main species in the system. There are two options: a more complex model that includes a fourth state variable to account for the dynamics of light irradiance acclimation, I^* ; or a simpler model using only three state variables – X , Q , and S . While the latter may have a lighter take on the time-dependent influence of light irradiance acclimation on microalgae growth – as the growth rate is a function of this factor – but it facilitates and allows a more straightforward implementation. For this reason, the three-state approach is chosen.

$$\dot{X} = [\mu(X, Q, I_0) - R]X - DX \quad 353$$

$$\dot{Q} = \rho(Q, S) - \mu(X, Q, I_0, Chl)(Q - Q_0) \quad 354$$

$$\dot{S} = D(S_{in} - S) - \rho(Q, S)X \quad 355$$

At this instance, the proposed model is governed by three state variables – biomass (X), internal nutrient quota (Q), and substrate concentration (S) – with two neural networks approximating key biological rates: growth rate (μ) and nutrient uptake rate (ρ). These learned rates are integrated into the predefined mass balance equations, preserving mechanistic structure while leveraging data-driven nonlinear function approximation. In this way, the model includes species X and S , while still considering important dynamics concerning the accumulation of nutrients as Q .

$$NN_1(X, Q, I_0, \theta_1) = \mu(X, Q, I_0, Chl) - R \quad 356$$

$$NN_2(Q, S, \theta_2) = \rho(Q, S) \quad 357$$

The Respiration Rate, R , is also absorbed into the NN_1 sub-model, as its constant nature acts as a mere bias on the growth rate. This action allows a hybrid model free of any physical parameters, focusing the training effort on the neural networks only. The final expression of the NN-based hybrid model becomes as follows:

$$\begin{bmatrix} \dot{X} \\ \dot{Q} \\ \dot{S} \end{bmatrix} = \begin{bmatrix} NN_1(X, Q, I_0, \theta_1) X - DX \\ NN_2(Q, S) - NN_1(X, Q, I_0, \theta_1) (Q - Q_0) \\ D(S_{in} - S) - NN_2(Q, S, \theta_2) X \end{bmatrix} \quad 358$$

Networks' parameters θ_1 and θ_2 refer to the weights (w_{ij}), biases (b_{ij}), and any additional parameters influencing the activation functions within the neurons of each network.

These equations still have a general basic mechanistic structure that we know to be true based on the principle of material conservation. The neural networks are embedded within this structure to model the kinetics of growth and substrate absorption and, in particular, to account for the influence of light irradiance on the growth rate. Notice that now the influence of the two inputs is decoupled, with the light irradiance being present only in the neural network related to the growth rate, while the dilution rate is only part of the mechanistic expressions.

Figure 83 illustrates the procedure employed to integrate the model equations over time. At a given time instance j , the physics-based model, depicted in light grey, provides the system state information, x_j , to the neural network, shown in dark grey. The neural network subsequently utilizes this information in conjunction with the system inputs, u_j , to estimate the internal model variables, ϕ_j , specifically the kinetics μ and ρ . Thereafter, a time integration process is executed to predict the state variables for the subsequent time instant. This iterative cycle is maintained throughout the entire simulation period.

For the implementation of this hybrid model, a similar procedure to the full neural network model is implemented. With fully connected layers and feedforward architecture, two neural networks are defined: NN_1 (μ estimator): 3-layer network (3-8-8-1 nodes) using X , Q , and I_0 as inputs; and NN_2 (ρ estimator): 3-layer network (2-8-8-1 nodes) using Q and S as inputs. LeakyReLU activation functions ($\alpha = 0.1$) are present after each hidden layer. To give a

dimension of the problem, the first neural network has a total of 113 parameters, while the second one presents 105 parameters.

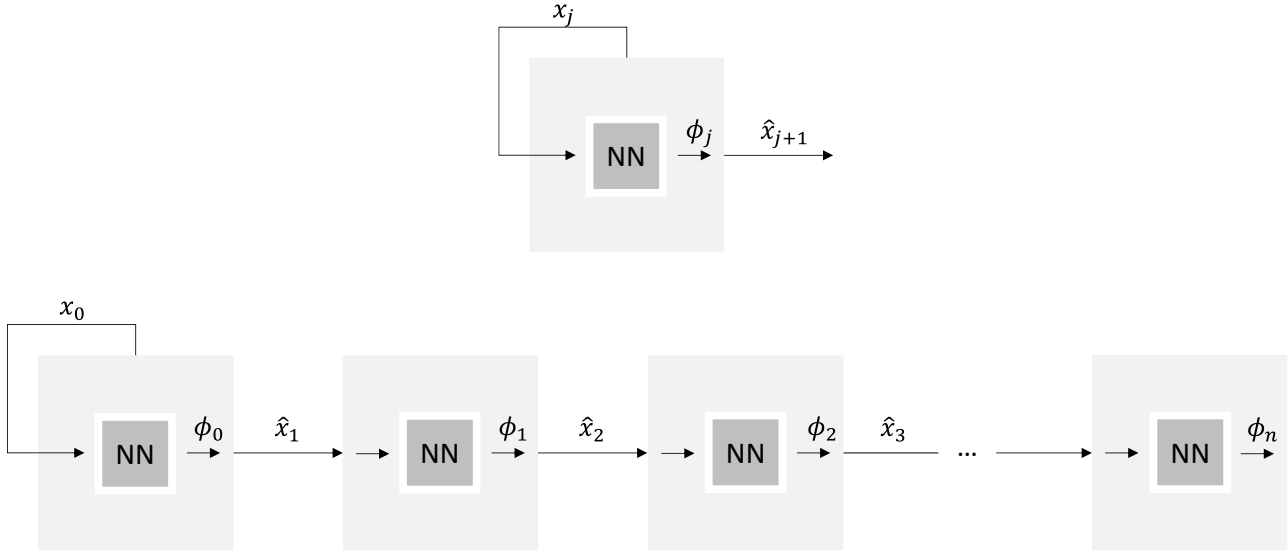


Figure 83. NN-based Hybrid Model Time Instant Simulation Diagram.

At a given time instant j , the physics-based model (light grey) supplies the neural network (dark grey) with the system states information x_j . The neural network then uses this information, along with the system inputs u_j , to estimate internal model variables (ϕ_j), specifically the kinetics μ and ρ . Subsequently, a time integration process predicts the states for the next time instant. This cycle is repeated throughout the entire timeframe.

TARGET ESTIMATION AND MEASUREMENTS SMOOTHING

Given the presence of noise, the measurements are smoothed by the use of a moving average. There are two options: to apply the moving average to the measurements prior to computing the derivatives – i.e., dX/dt and dS/dt ; or to compute the derivatives with noise and then smooth the measurements and derivatives apart.

In numerical differentiation, noise is typically amplified during the computation of derivatives. Therefore, smoothing the measurements beforehand can reduce the noise level, potentially leading to more accurate derivative estimates. However, pre-smoothing may also introduce bias in the signal, which might affect the accuracy of the derivative if the system dynamics are highly sensitive or fast-changing. In this case, as the noise is significant and the dynamics are slow, smoothing the measurements first is preferred.

Measurements of a feature x are smoothed with the following equation, with padding at the beginning and end of the timeframe:

$$x_{smoothed}|_i = \frac{1}{w} \sum_{j=-w/2}^{w/2} x(i+j) \quad 359$$

where i is the time instant under consideration; and w defines the window size under consideration; in this case, $w = 2$. After the measurements smoothing step, first-principles derivatives are computed as:

$$\left. \frac{dx}{dt} \right|_i = \frac{x_{smoothed}|_{i-1} - x_{smoothed}|_i}{\Delta t} \quad 360$$

Later on, the target data for μ and ρ is calculated with expressions algebraically derived from Equations 353-355. In this way, the kinetic rates are estimated by only using measurements of X and S – no need for measurements of Q .

$$\bar{\mu}_i = \frac{(dX/dt|_i)}{X} + D \quad 361$$

$$\bar{\rho}_i = \frac{[D(S_{in} - S) - dS/dt|_i]}{X} \quad 362$$

MODEL TRAINING

The inputs to the neural network – or features – and their targets present different scale values, necessitating a normalization process to bring them all to the same scale. Before feeding the data into the neural network, each feature and target variable is normalized. Normalization typically involves scaling the data to a standard range, such as $[0, 1]$ or $[-1, 1]$, or transforming it to have a mean of zero and a standard deviation of a unit. This can be achieved using methods such as min-max scaling or z-score normalization. The normalization process ensures that the neural network treats all inputs equally, preventing features with larger scales from dominating the learning process. Here, a z-score normalization is applied.

Mathematically, for z-score normalization, the normalized value x norm of a given feature can be computed as:

$$x_{normalized} = \frac{x - \mu}{\sigma} \quad 363$$

where μ and σ are the mean and standard deviation of the feature x , respectively.

The model is trained using normalized data to address scale disparities between the variables, with the mean squared error serving as the loss function. To evaluate the model generalization capabilities, K-fold cross-validation is utilized across five training batches, while the sixth batch is designated as an external test set. Training is performed using the ADAM optimization algorithm with a learning rate of 10^{-3} over 20,000 epochs.

With normalized inputs, the neural network is trained to learn the relationships between the features and the targets. The training process involves adjusting the weights and biases of the network to minimize the error between the predicted and actual target values.

After the neural network is trained and predictions are made on new data, the predicted values are in the normalized scale. To interpret these predictions in their original context, a de-normalization process is applied. De-normalization reverses the normalization transformation, converting the predictions back to their original scale.

$$X_{de-normalized} = \sigma X_{normilized} + \mu \quad 364$$

This ensures that the predicted values can be correctly interpreted and compared with the original data.

To further improve predictive accuracy, an initial condition identification procedure was implemented after training in an iterative loop: model training followed by initial conditions identification; after a few iterations of this loop, a convergence point is reached. This optimization process adjusted the initial conditions (X_0, Q_0, S_0) for each batch using gradient descent, minimizing the discrepancies between the simulated trajectories and the measured data points. The optimization was conducted according to constraints of $\pm 5\%$ around the nominal values for the initial conditions as measured experimentally.

The results of the whole process, including data pre-processing, model training, and predictions, are shown in Figures 85-87 as a comparison between the NN-based hybrid model predictions versus the real data measurements for all the states. The first figure contains the

prediction for a training batch – the third batch, while the second figure contains the predictions for the cross-validation batch – the sixth batch.

Figure 84 contains the direct prediction outputs from the neural networks, NN_1 and NN_2 , versus the training targets. As observed, the neural networks replicate the target values with a satisfactory level of accuracy, and then this leads to a good level of correspondence between the hybrid model predictions and the real system measurements.

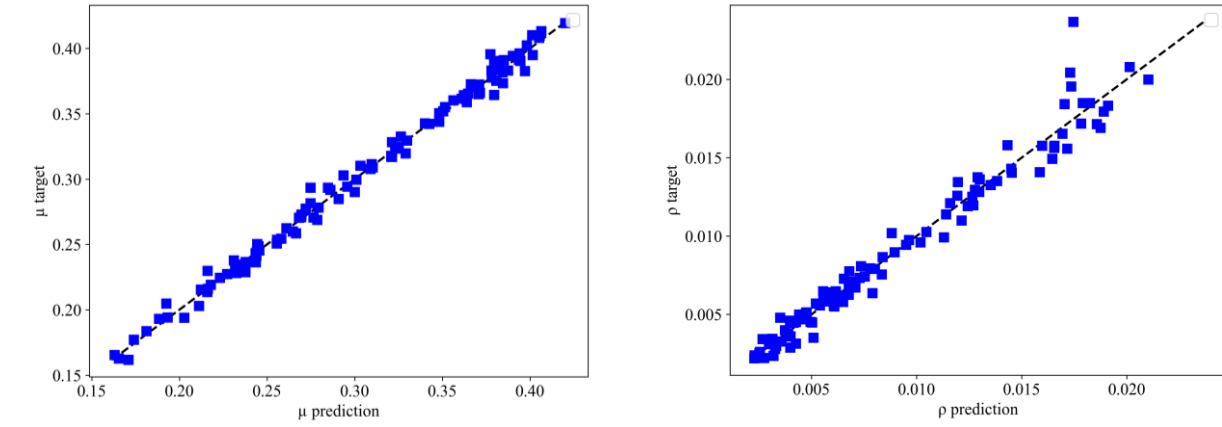


Figure 84. Predicted versus Actual Kinetic Targets for Neural Networks Performance.

Scatter plots illustrating the performance of NN_1 (left) and NN_2 (right) in predicting kinetic targets. The close alignment of the data points along the diagonal indicates a high degree of predictive accuracy for both models.

NN_1 Accuracy in predicting μ target:

Mean Squared Error: 0.000036; Mean Relative Error: 1.60%.

NN_2 Accuracy in predicting ρ target:

Mean Squared Error: 0.000001; Mean Relative Error: 8.12%.

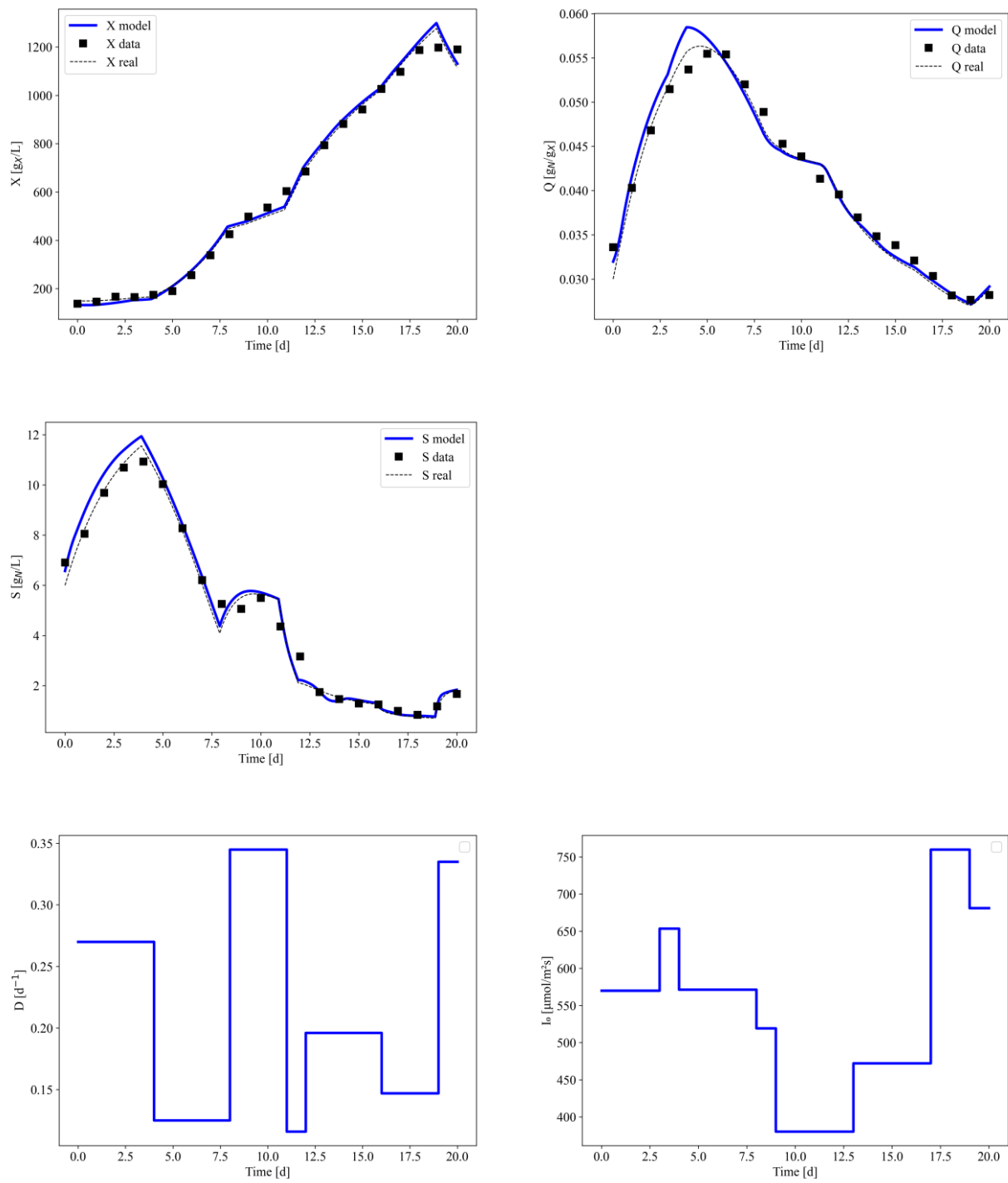


Figure 85. Microalgae Culture Synthetic Data versus NN-based Hybrid Model Predictions (model training). Synthetic data generated with a physics-based mathematical model is compared against the predictions made by the NN-based hybrid model.

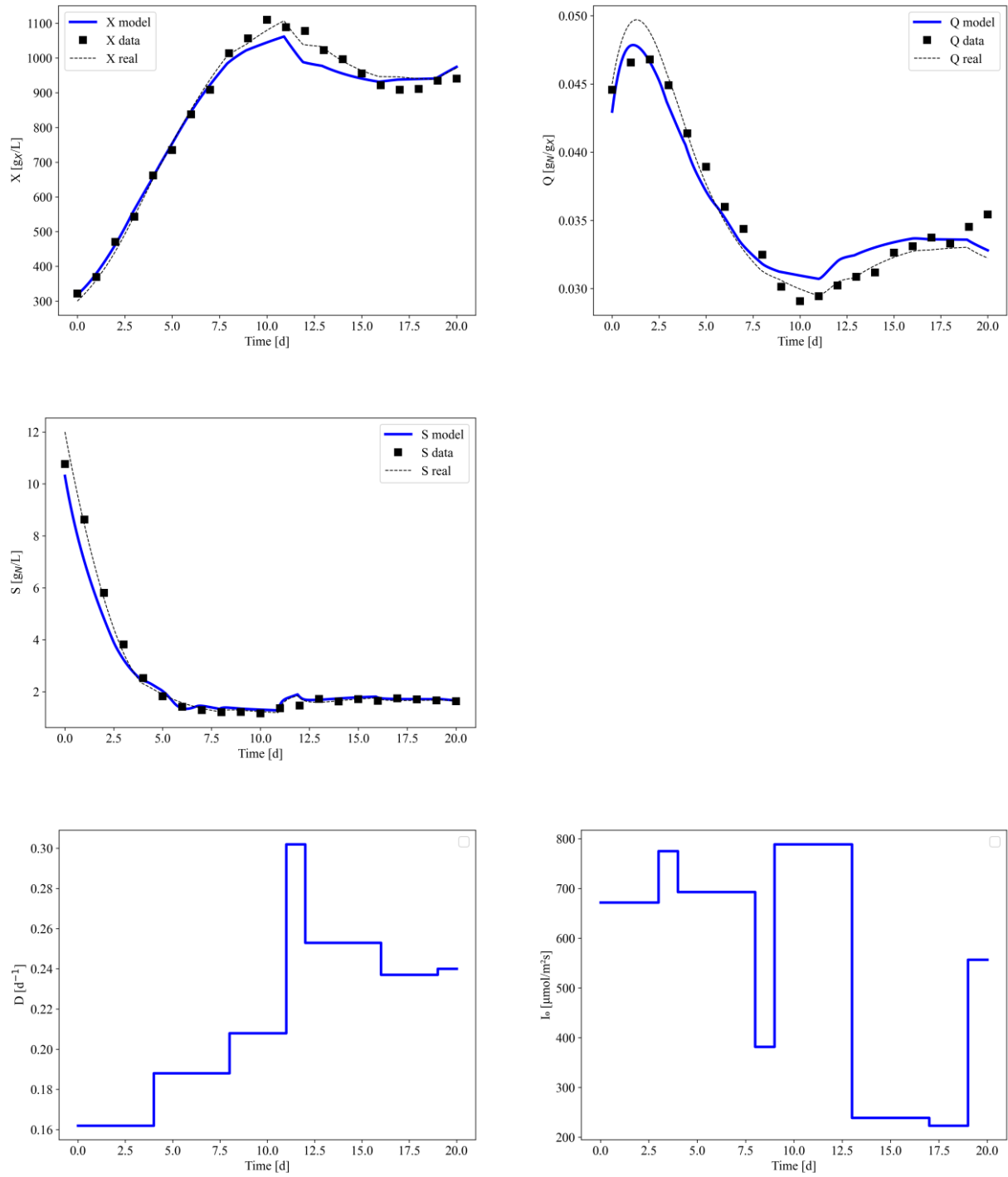


Figure 86. Microalgae Culture Synthetic Data versus NN-based Hybrid Model Predictions (model training). Synthetic data generated with a physics-based mathematical model is compared against the predictions made by the NN-based hybrid model.

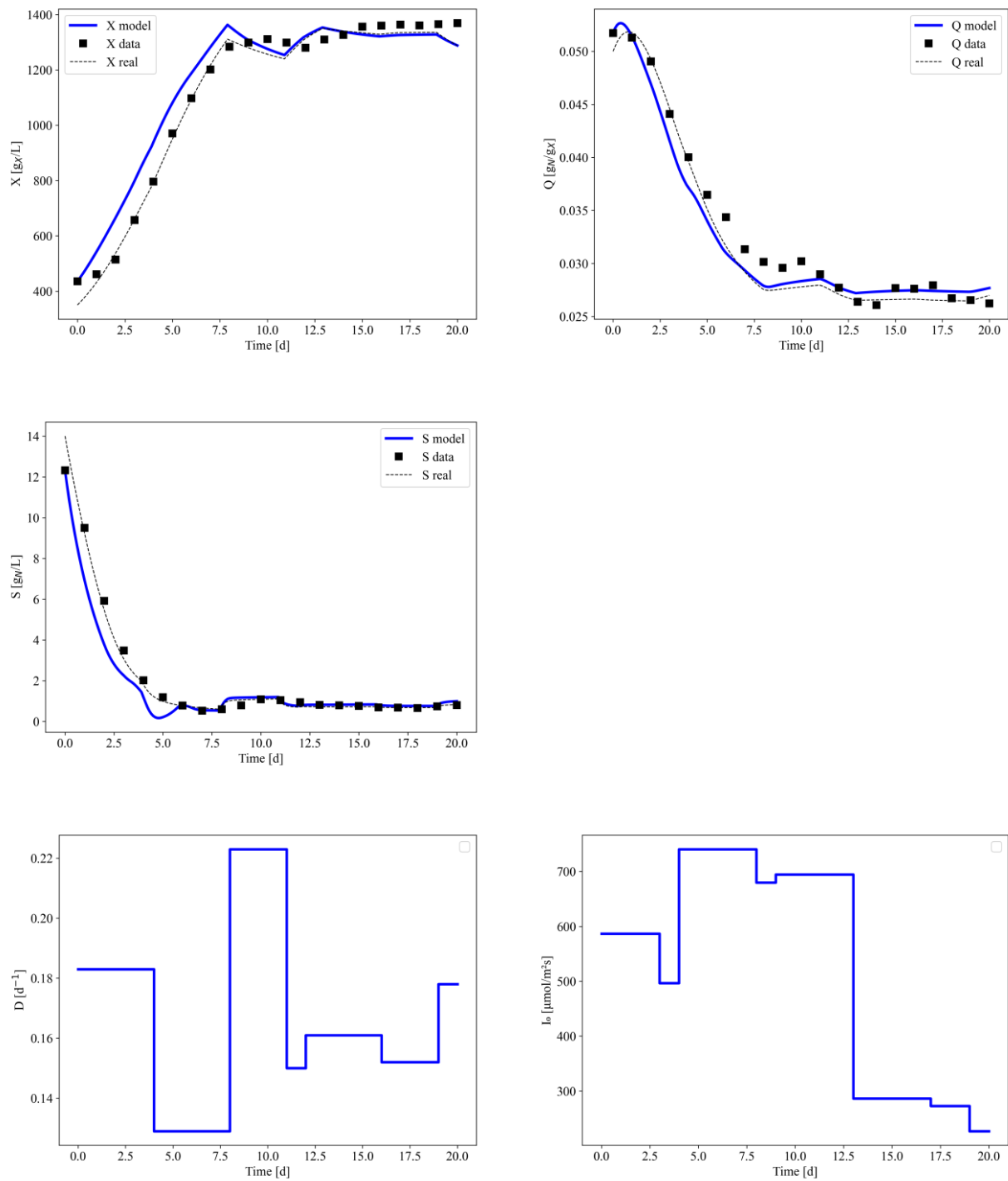


Figure 87. Microalgae Culture Synthetic Data versus NN-based Hybrid Model Predictions (model cross-validation).

Synthetic data generated with a physics-based mathematical model is compared against the predictions made by the NN-based hybrid model.

Annex G contains an overview of the parameters – weights and biases – for the trained neural networks NN_1 and NN_2 .

Although the model was subjected to challenging conditions that tested its capacity to capture the system behavior, it still demonstrated robust performance in replicating the system outputs. In the worst-case scenario – when different input values for D and I_0 were imposed at each sampling time – the neural network's predictive capability deteriorated significantly. This outcome is understandable, as the network did not have the opportunity to observe the system evolution under a consistent input. Nonetheless, the model performed satisfactorily, yielding predictions within acceptable bounds.

Table 23. Hybrid Model Predictions Relative Errors (Hybrid Model versus Noisy Measurements).

Batch		$X_{relative\ error}$	$Q_{relative\ error}$	$S_{relative\ error}$
1	Training	0.071	0.023	0.112
2	Training	0.058	0.030	0.081
3	Training	0.030	0.025	0.118
4	Training	0.097	0.061	0.164
5	Training	0.029	0.035	0.087
6	Validation	0.072	0.040	0.201

Model predictions are computed against measurement data (Table 23) and also against the real system data (Table 24) – this latter information is not known by the hybrid model while training. The relative errors against measurement data are generally in line with the level of measurement noise, which is considerable – 4% of the state mean. When comparing the predictions against the real data, these relative errors drop, showing a high correspondence between the predictions and the system.

Table 24. Hybrid Model Predictions Relative Errors (Hybrid Model versus Real System).

Batch		$X_{relative\ error}$	$Q_{relative\ error}$	$S_{relative\ error}$
1	Training	0.047	0.022	0.105
2	Training	0.035	0.018	0.083
3	Training	0.028	0.012	0.112
4	Training	0.084	0.045	0.164
5	Training	0.028	0.031	0.080
6	Validation	0.074	0.030	0.201

As soon as the conditions are set up to favor model training – e.g., maintaining the input value for multiple sampling times – the accuracy of the neural networks improves drastically, as well as the hybrid model’s overall performance.

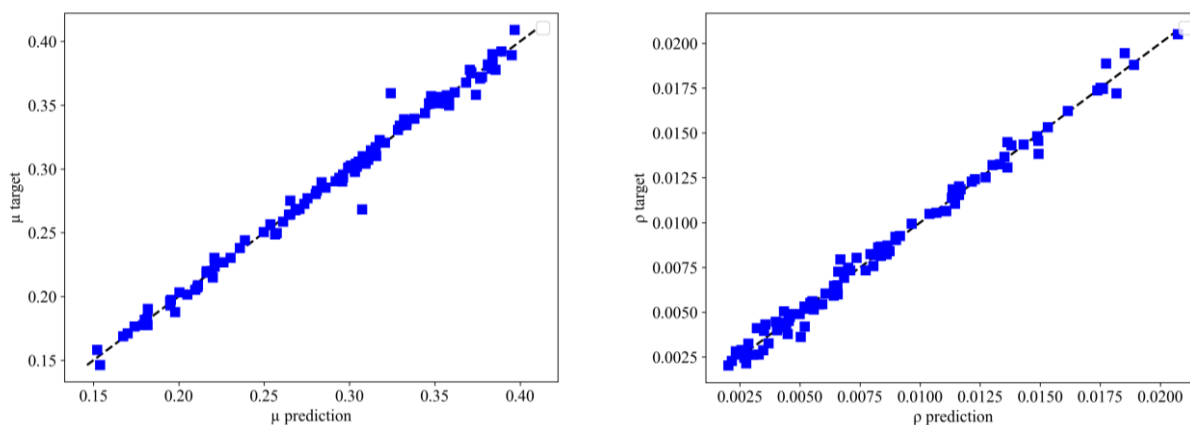


Figure 88. Predicted versus Actual Kinetic Targets for Neural Networks Performance.

Scatter plots illustrating the performance of NN_1 (left) and NN_2 (right) in predicting kinetic targets. The close alignment of the data points along the diagonal indicates a high degree of predictive accuracy for both models.

NN1 Accuracy in predicting μ target:

Mean Squared Error: 0.000047; Mean Relative Error: 1.46%.

NN2 Accuracy in predicting ρ target:

Mean Squared Error: 0.000000; Mean Relative Error: 5.45%.

For instance, in Figures 89-91, the simulation results of training the model under these favorable conditions are shown, with periods of constant input equal to four sampling times. Figure 88 provides a clear view of the particular performance of the neural networks, which now has a high level of accuracy, achieving good prediction on the targets μ and ρ – although now showing a slight sign of deterioration in the accuracy, which can indicate more difficulties from the neural networks in capturing the behavior of the internal kinetic variables.

The NN-based hybrid model employs a multi-step forecasting strategy, capturing the temporal dynamics of the culture over an extended period.

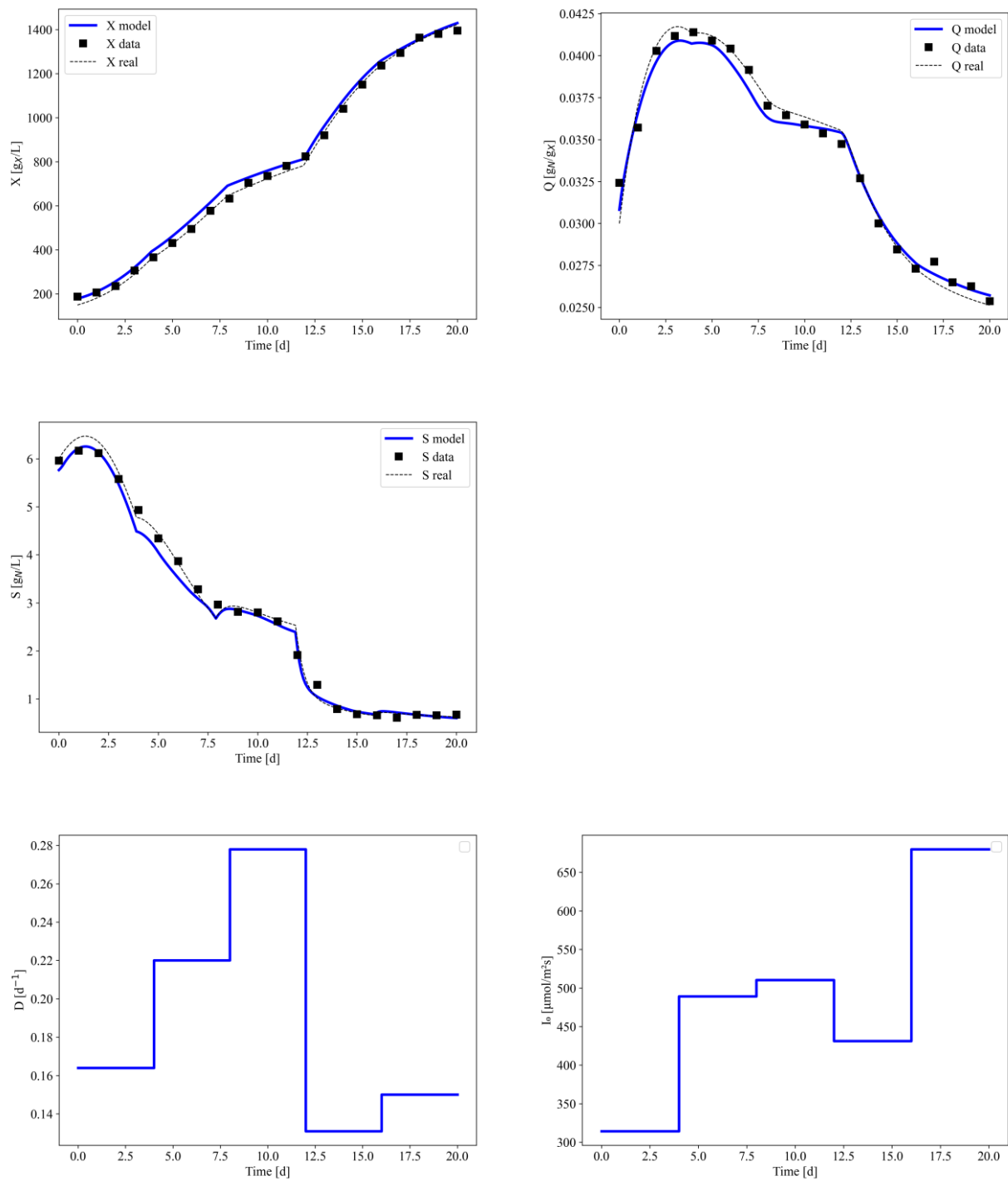


Figure 89. Microalgae Culture Synthetic Data versus NN-based Hybrid Model Predictions (model training).

Synthetic data generated with a physics-based mathematical model is compared against the predictions made by the NN-based hybrid model.

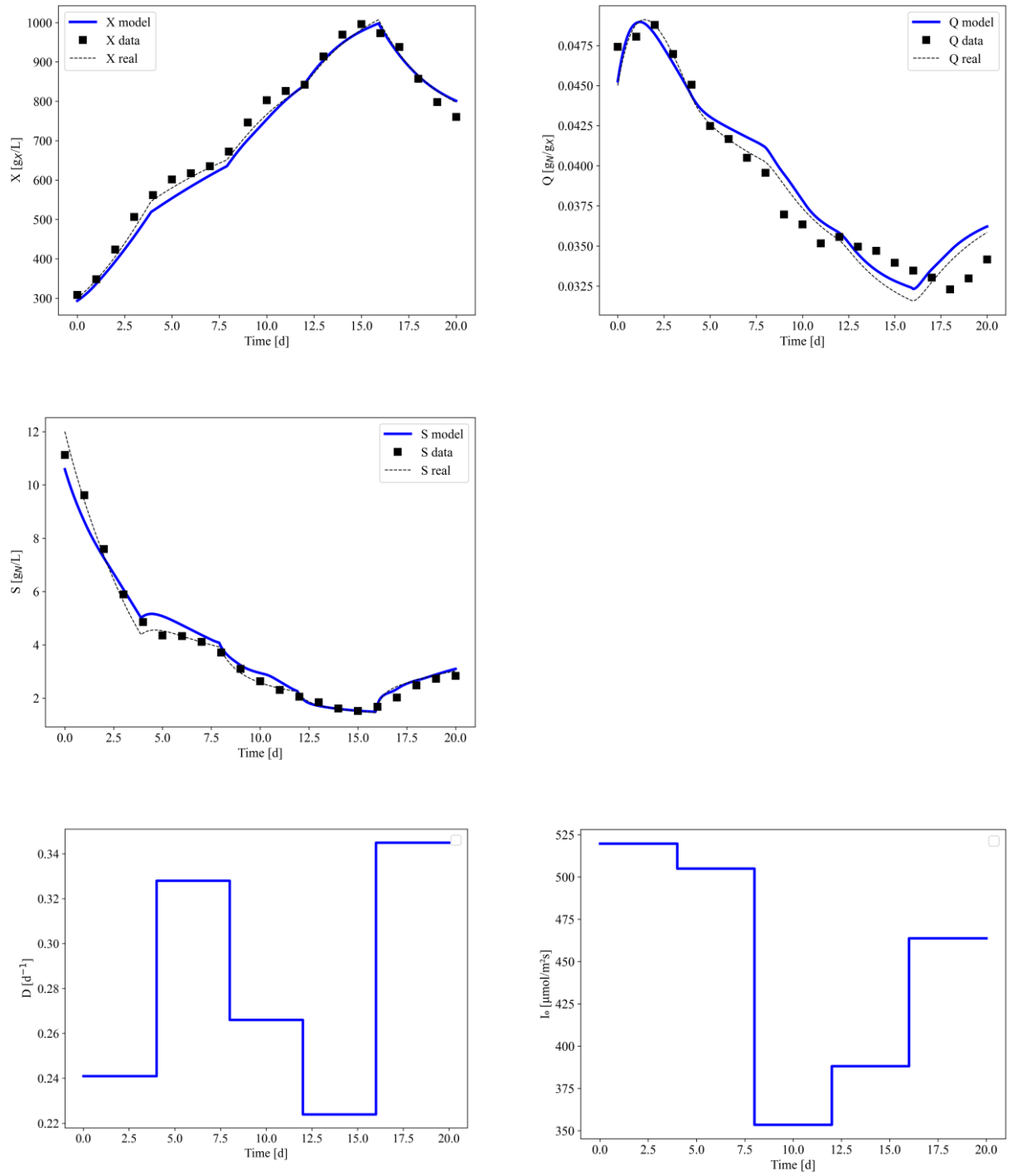


Figure 90. Microalgae Culture Synthetic Data versus NN-based Hybrid Model Predictions (model training). Synthetic data generated with a physics-based mathematical model is compared against the predictions made by the NN-based hybrid model.

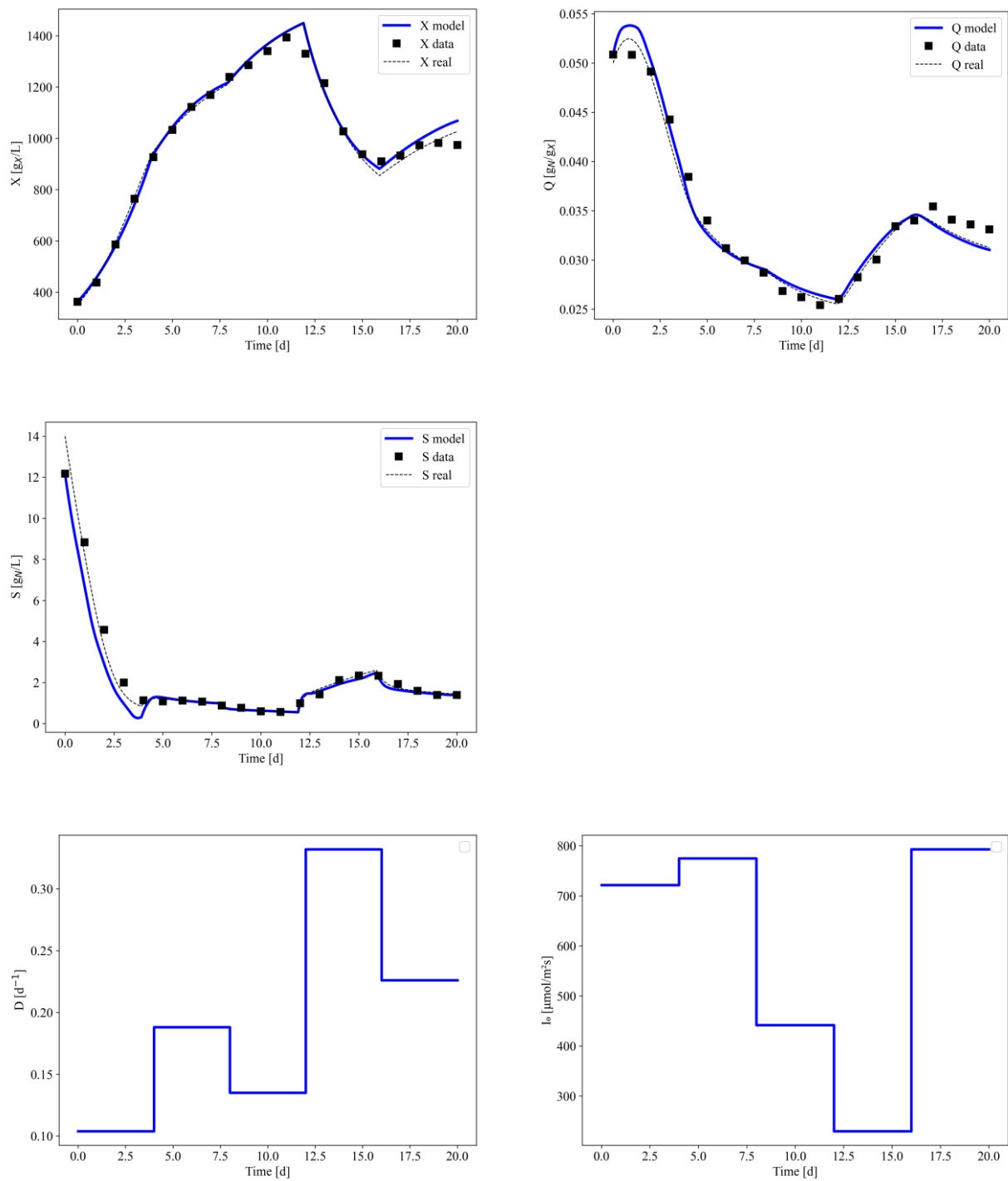


Figure 91. Microalgae Culture Synthetic Data versus NN-based Hybrid Model Predictions (model cross-validation).

Synthetic data generated with a physics-based mathematical model is compared against the predictions made by the NN-based hybrid model.

As a final thought, although this proposal has a good performance, there might be instances where more structure from the mechanistic model is desired. Monod and Droop kinetics for substrate absorption and internal quota are well-established relationships. Hence, other hybrid models with further use of mechanistic dynamics can be built.

Both substrate presence and light irradiance impact growth rate; the substrate internal quota is a dominant and clear factor, which means that Droop kinetics can be retained; this only leaves room for data-driven modelling of the effects of light on the growth rate.

$$\mu(X, Q, I_0, Chl) = \bar{\mu} \left(1 - \frac{Q_0}{Q}\right) NN_1(X, Q, I_0, \theta_1) \quad 365$$

$$\rho(Q, S) = NN_2(Q, S, \theta_2) \quad 366$$

Replacing these neural networks, the final expression of the hybrid macroscopic and data-driven model becomes as follows:

$$\dot{X} = \bar{\mu} \left(1 - \frac{Q_0}{Q}\right) NN_1(X, Q, I_0, \theta_1) X - DX - RX \quad 367$$

$$\dot{Q} = NN_2(Q, S, \theta_2) - \bar{\mu} \left(1 - \frac{Q_0}{Q}\right) NN_1(X, Q, I_0, \theta_1) (Q - Q_0) \quad 368$$

$$\dot{S} = DS_{in} - NN_2(Q, S, \theta_2) X - DS \quad 369$$

However, this model necessitates not only the training of data-driven sub-models but also the identification of several physics-based parameters.